ITM UNIVERSITY • DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

AZURA TOURIST PLATFORM

Formal Normalization Worksheets (0NF → 1NF → 2NF → 3NF → BCNF)
& Complete Relational Schema Mapping

Course / Subject	Database Platform	Team / Project	Faculty In-Charge
CSL0311: DBMS	MySQL 8.0 (InnoDB)	Team NEXUS (Kush Tayal, Krishna Yadav, Aniket Yadav)	Dr. Geetanjali Surange

1. Complete Logical Relational Schema Mapping

The conceptual Chen ER diagram maps directly to the following 16 normalized relational schemas. Primary Keys (PK) are underlined, and Foreign Keys (FK) are marked with asterisks (*) referencing their parent candidate keys.

1. USERS: USERS (User_ID [PK], Name, Email [UNIQUE], Phone, Password, Role)

2. DESTINATIONS: DESTINATIONS (Destination_ID [PK], Name, Country, Rating, Price, Description)

3. HOTELS: HOTELS (Hotel_ID [PK], Destination_ID* [FK], Hotel_Name, Stars, Price, Amenities)
FOREIGN KEY (Destination_ID) REFERENCES DESTINATIONS(Destination_ID) ON DELETE CASCADE

4. ROOMS (Weak Entity): ROOMS (Hotel_ID* [PK, FK], Room_Number [PK], Room_Type, Price_Per_Night, Availability)
FOREIGN KEY (Hotel_ID) REFERENCES HOTELS(Hotel_ID) ON DELETE CASCADE

5. AMENITIES: AMENITIES (Amenity_ID [PK], Amenity_Name [UNIQUE])

6. HOTEL_AMENITIES (M:N Bridge): HOTEL_AMENITIES (Hotel_ID* [PK, FK], Amenity_ID* [PK, FK])
FOREIGN KEY (Hotel_ID) REFERENCES HOTELS(Hotel_ID),
FOREIGN KEY (Amenity_ID) REFERENCES AMENITIES(Amenity_ID)

7. PACKAGES: PACKAGES (Package_ID [PK], Destination_ID* [FK], Package_Name, Duration, Price, Travel_Style, Rating)
FOREIGN KEY (Destination_ID) REFERENCES DESTINATIONS(Destination_ID) ON DELETE CASCADE

8. EXPERIENCES: EXPERIENCES (Experience_ID [PK], Destination_ID* [FK], Title, Category, Duration, Price, Rating)
FOREIGN KEY (Destination_ID) REFERENCES DESTINATIONS(Destination_ID) ON DELETE CASCADE

9. PACKAGE_EXPERIENCES (M:N Bridge): PACKAGE_EXPERIENCES (Package_ID* [PK, FK], Experience_ID* [PK, FK])
FOREIGN KEY (Package_ID) REFERENCES PACKAGES(Package_ID),
FOREIGN KEY (Experience_ID) REFERENCES EXPERIENCES(Experience_ID)

10. BOOKINGS: BOOKINGS (Booking_ID [PK], User_ID* [FK], Package_ID* [FK], Booking_Date, Number_Of_Travelers, Total_Amount, Booking_Status)
FOREIGN KEY (User_ID) REFERENCES USERS(User_ID),
FOREIGN KEY (Package_ID) REFERENCES PACKAGES(Package_ID)

11. PAYMENTS: PAYMENTS (Payment_ID [PK], Booking_ID* [FK, UNIQUE], Payment_Method, Amount, Payment_Date, Payment_Status, Transaction_ID [UNIQUE])
FOREIGN KEY (Booking_ID) REFERENCES BOOKINGS(Booking_ID)

12. REVIEWS: REVIEWS (Review_ID [PK], User_ID* [FK], Destination_ID* [FK], Rating, Review_Text, Review_Date)
FOREIGN KEY (User_ID) REFERENCES USERS(User_ID),
FOREIGN KEY (Destination_ID) REFERENCES DESTINATIONS(Destination_ID)

13. AUDIT_LOGS: AUDIT_LOGS (Audit_ID [PK], User_ID*, Action, Table_Name, Record_ID, Old_Data [JSON], New_Data [JSON], Changed_At)

2. Normalization Worksheets (Step-by-Step Formal Proof)

Normalization is the systematic approach of decomposing tables to eliminate data redundancy and prevent insertion, update, and deletion anomalies. Below are two comprehensive case studies demonstrating functional dependency analysis from 0NF to BCNF.

Worksheet Case Study 1: Accommodation & Room Management

Domain Scenario: A guest accommodation record tracking hotels, destination locations, room configurations, and individual room rates.

Step 0: Unnormalized Form (UNF / 0NF)

Relation R_UNF contains repeating groups (multiple rooms per hotel) and composite multivalued attributes (amenities list):

R_UNF (Hotel_ID, Hotel_Name, Destination_ID, Destination_Name, Country, { Room_Number, Room_Type, Price_Per_Night, Availability }, { Amenity_Name })

Anomalies in 0NF: Attributes contain non-atomic collections and nested repeating rows, violating relational algebra foundational definitions.

Step 1: First Normal Form (1NF)

1NF Rule: All attribute values must be atomic (indivisible scalars), repeating groups must be eliminated, and a primary key must be defined.

Transformation: Flatten repeating groups by creating separate tuples for each room and amenity. Candidate Key becomes composite: { Hotel_ID, Room_Number, Amenity_Name }.

Hotel_ID	Hotel_Name	Dest_ID	Room_No	Room_Type	Price/Night	Amenity
H101	Santorini Cliff Suites	D1	R101	Deluxe	$3,500	WiFi
H101	Santorini Cliff Suites	D1	R101	Deluxe	$3,500	Pool
H101	Santorini Cliff Suites	D1	R102	Suite	$4,800	WiFi
H102	Mykonos Blue Resort	D1	R201	Ocean View	$2,900	Spa

1NF Functional Dependencies (FDs):

• FD1: {Hotel_ID, Room_Number} → {Room_Type, Price_Per_Night, Availability}
• FD2: {Hotel_ID} → {Hotel_Name, Destination_ID, Destination_Name, Country}
• FD3: {Destination_ID} → {Destination_Name, Country}
• FD4: {Amenity_ID} → {Amenity_Name}

Step 2: Second Normal Form (2NF)

2NF Rule: Must be in 1NF AND contain NO Partial Functional Dependencies (every non-prime attribute must depend on the WHOLE candidate key, not a proper subset).

Identified Anomaly in 1NF (Partial Dependencies):

In candidate key {Hotel_ID, Room_Number}, attribute Hotel_Name depends strictly on Hotel_ID alone (FD2), completely independent of Room_Number! This causes massive redundancy and update anomalies whenever hotel names change.

2NF Decomposition: Decompose into relations having full functional dependency on their respective primary keys:

• R1_HOTEL (Hotel_ID [PK], Hotel_Name, Destination_ID, Destination_Name, Country)
• R2_ROOM (Hotel_ID* [PK, FK], Room_Number [PK], Room_Type, Price_Per_Night, Availability)
• R3_HOTEL_AMENITY (Hotel_ID* [PK, FK], Amenity_ID* [PK, FK])

Step 3: Third Normal Form (3NF)

3NF Rule: Must be in 2NF AND contain NO Transitive Dependencies (non-prime attributes must not determine other non-prime attributes: X → Y where Y is non-prime and X is not a superkey).

Identified Anomaly in 2NF (Transitive Dependency in R1_HOTEL):

• Hotel_ID → Destination_ID (Primary Key determines Destination_ID)
• Destination_ID → {Destination_Name, Country} (Destination_ID determines Country)
• Therefore: Hotel_ID → {Destination_Name, Country} is a TRANSITIVE DEPENDENCY via Destination_ID!

Consequences: If country spelling is updated, multiple hotels in that destination must be redundantly altered. If all hotels in a destination are deleted, the destination name is lost (Deletion Anomaly).

3NF Decomposition (Lossless Join & Dependency Preserving):

• HOTELS (Hotel_ID [PK], Destination_ID* [FK], Hotel_Name)
• DESTINATIONS (Destination_ID [PK], Name, Country)
• ROOMS (Hotel_ID* [PK, FK], Room_Number [PK], Room_Type, Price_Per_Night, Availability)
• AMENITIES (Amenity_ID [PK], Amenity_Name)
• HOTEL_AMENITIES (Hotel_ID* [PK, FK], Amenity_ID* [PK, FK])

Step 4: Boyce-Codd Normal Form (BCNF)

BCNF Rule: A relation is in BCNF if for every non-trivial functional dependency X → Y, determinant X is a SUPERKEY.

BCNF Verification:

• In HOTELS: Determinant is {Hotel_ID}, which is the Primary Key (Superkey) → BCNF Satisfied ☑
• In DESTINATIONS: Determinant is {Destination_ID}, which is the Primary Key (Superkey) → BCNF Satisfied ☑
• In ROOMS: Determinant is {Hotel_ID, Room_Number}, which is the Composite PK (Superkey) → BCNF Satisfied ☑
• In AMENITIES: Determinant is {Amenity_ID}, which is the Primary Key (Superkey) → BCNF Satisfied ☑
Conclusion: All decomposed tables satisfy BCNF with zero functional dependency loss.

Worksheet Case Study 2: Reservation & Billing Lifecycle

Domain Scenario: A customer tour package reservation record tracking traveler details, package itineraries, calculated prices, and transactional payments.

Step 0 & 1NF: Unnormalized to First Normal Form

Initial combined booking record R_BOOKING_UNF:

R_BOOKING (Booking_ID, User_ID, User_Name, User_Email, Package_ID, Package_Name, Package_Price, Destination_ID, Destination_Name, Booking_Date, Travelers, Total_Amount, Payment_ID, Payment_Method, Transaction_ID, Payment_Status)

1NF Enforced: Primary Key is {Booking_ID}. Candidate Key is {Booking_ID} and alternative candidate key is {Transaction_ID}.

Step 2: Second Normal Form (2NF)

Since the Primary Key {Booking_ID} is a single-column atomic identifier, NO partial dependencies can mathematically exist on a proper subset of the key. Therefore, R_BOOKING is automatically in 2NF.

Step 3: Third Normal Form (3NF Analysis & Decomposition)

Examine Functional Dependencies in R_BOOKING:

- FD1: Booking_ID → {User_ID, Package_ID, Booking_Date, Travelers, Total_Amount}
- FD2: User_ID → {User_Name, User_Email} [Transitive Dependency on non-superkey User_ID]
- FD3: Package_ID → {Package_Name, Package_Price, Destination_ID} [Transitive Dependency on Package_ID]
- FD4: Destination_ID → {Destination_Name} [Transitive Dependency on Destination_ID]
- FD5: Booking_ID → {Payment_ID, Payment_Method, Transaction_ID, Payment_Status}
- FD6: Transaction_ID → {Payment_ID, Booking_ID, Amount, Payment_Status}

Decomposition into 3NF Relations:

1. USERS (User_ID [PK], Name, Email [UNIQUE]) — isolates user profile facts.
2. PACKAGES (Package_ID [PK], Destination_ID* [FK], Package_Name, Price) — isolates tour facts.
3. DESTINATIONS (Destination_ID [PK], Name) — isolates destination catalog facts.
4. BOOKINGS (Booking_ID [PK], User_ID* [FK], Package_ID* [FK], Booking_Date, Travelers, Total_Amount) — transactional fact table.
5. PAYMENTS (Payment_ID [PK], Booking_ID* [FK, UNIQUE], Payment_Method, Transaction_ID [UNIQUE], Payment_Status) — reconciliation fact table.

Step 4: Boyce-Codd Normal Form (BCNF Proof)

Evaluate every determinant across all 3NF decomposed relations:

Relation	Functional Dependency	Determinant (X)	Is Superkey?	BCNF Status
USERS	User_ID → {Name, Email}	User_ID	YES (PK)	PASS (BCNF)
PACKAGES	Package_ID → {Name, Price,	Package_ID	YES (PK)	PASS (BCNF)

	Dest_ID}			
DESTINATIONS	Destination_ID → {Name, Country}	Destination_ID	YES (PK)	PASS (BCNF)
BOOKINGS	Booking_ID → {User, Pkg, Date, Amount}	Booking_ID	YES (PK)	PASS (BCNF)
PAYMENTS	Payment_ID → {Booking, Amount, Txn}	Payment_ID	YES (PK)	PASS (BCNF)
PAYMENTS	Transaction_ID → {Payment_ID, Booking}	Transaction_ID	YES (Candidate)	PASS (BCNF)

Mathematical Conclusion: Because every determinant across all relations is a valid Superkey (Candidate Key), the entire AZURA database schema satisfies Boyce-Codd Normal Form (BCNF) with zero anomalies and 100% dependency preservation.

3. Summary of Anomalies Resolved by Normalization

Anomaly Type	Pre-Normalization State (0NF/1NF)	Post-Normalization State (3NF/BCNF)	Business Benefit in AZURA
Insertion Anomaly	Cannot add a new Destination without creating an artificial Booking or Hotel record.	Destinations can be added independently into DESTINATIONS table.	Marketing team can seed 100+ global destinations prior to any traveler bookings.
Deletion Anomaly	Deleting the last booking for a customer removes all customer contact records from the system.	Deleting a booking from BOOKINGS leaves the customer profile intact in USERS.	Customer retains their account history, loyalty tier, and login credentials.
Update Anomaly	Changing package price requires updating every historical booking row containing that package.	Package price is updated once in PACKAGES; historical booking amounts stay locked in BOOKINGS.	Financial integrity preserved; past transactions do not mutate when current catalog prices adjust.